Business Queries Solved

1. Which products are driving revenue?

```
SELECT
    product_name,
    SUM(sales) AS total_sales
FROM superstore.orders
GROUP BY product_name
ORDER BY total_sales DESC;
```

---

2. Which regions and states are performing best? By Region

```
SELECT
    region,
    SUM(sales) AS total_sales
FROM superstore.orders
GROUP BY region
ORDER BY total_sales DESC;
```

By State

```
SELECT
    state,
    SUM(sales) AS total_sales
FROM superstore.orders
GROUP BY state
ORDER BY total_sales DESC;
```

---

3. Which customer segments are most valuable?

```
SELECT
    segment,
    COUNT(*) AS total_orders,
    SUM(quantity) AS total_quantity,
    SUM(sales) AS total_sales,
```

```
    AVG(sales) AS avg_sales
FROM superstore.orders
GROUP BY segment
ORDER BY total_sales DESC;
```

---

4. How are sales changing over time? Monthly Sales

```
SELECT
    year,
    month,
    SUM(sales) AS total_sales
FROM superstore.orders
GROUP BY year, month
ORDER BY year, MIN(order_date);
```

Daily Sales

```
SELECT
    order_date,
    SUM(sales) AS daily_sales
FROM superstore.orders
GROUP BY order_date
ORDER BY order_date;
```

---

5. Which categories and sub-categories contribute the most revenue?

Category

```
SELECT
    category,
    SUM(sales) AS total_sales
FROM superstore.orders
GROUP BY category
ORDER BY total_sales DESC;
```

Sub-Category

```
SELECT
    category,
```

```
    sub_category,
    SUM(sales) AS total_sales
FROM superstore.orders
GROUP BY category, sub_category
ORDER BY total_sales DESC;
```

---

6. Which products should management focus on?

Find the top 10 products based on sales:

```
SELECT
    product_name,
    SUM(sales) AS total_sales,
    SUM(quantity) AS total_quantity
FROM superstore.orders
GROUP BY product_name
ORDER BY total_sales DESC
LIMIT 10;
```

---

7. Where are the strongest sales opportunities?

Find the top-performing region + category combinations:

```
SELECT
    region,
    category,
    SUM(sales) AS total_sales
FROM superstore.orders
GROUP BY region, category
ORDER BY total_sales DESC;
```

Top 10 opportunities:

```
SELECT
    region,
    category,
    SUM(sales) AS total_sales
FROM superstore.orders
GROUP BY region, category
ORDER BY total_sales DESC
LIMIT 10;
```

---

8. Rank all products based on total sales

```
SELECT
    product_name,
    SUM(sales) AS total_sales,
    RANK() OVER (
        ORDER BY SUM(sales) DESC
    ) AS sales_rank
FROM superstore.orders
GROUP BY product_name
ORDER BY sales_rank;
```

---

9. Rank products within each category using `RANK()`

```
SELECT
    category,
    product_name,
    SUM(sales) AS total_sales,
    RANK() OVER (
        PARTITION BY category
        ORDER BY SUM(sales) DESC
    ) AS category_rank
FROM superstore.orders
GROUP BY category, product_name
ORDER BY category, category_rank;
```

---

10. Find the top 3 products within each category

```
WITH ranked_products AS (
    SELECT
        category,
        product_name,
        SUM(sales) AS total_sales,
        RANK() OVER (
            PARTITION BY category
            ORDER BY SUM(sales) DESC
        ) AS product_rank
    FROM superstore.orders
    GROUP BY category, product_name
)
```

```
SELECT
    category,
    product_name,
    total_sales,
```

---

11. Find the highest-selling product in each category

```
WITH ranked_products AS (
    SELECT
        category,
        product_name,
        SUM(sales) AS total_sales,
        ROW_NUMBER() OVER (
            PARTITION BY category
            ORDER BY SUM(sales) DESC
        ) AS rn
    FROM superstore.orders
    GROUP BY category, product_name
)

SELECT
    category,
    product_name,
    total_sales
FROM ranked_products
```

---

12. Calculate the running total of sales by order date

First calculate daily sales, then the running total:

```
WITH daily_sales AS (
    SELECT
        order_date,
        SUM(sales) AS daily_sales
    FROM superstore.orders
    GROUP BY order_date
)

SELECT
    order_date,
    daily_sales,
```

```
        SUM(daily_sales) OVER (
            ORDER BY order_date
        ) AS running_total_sales
    FROM daily_sales
    ORDER BY order_date;
```

---

13. Calculate monthly sales growth using `LAG()`

```
    WITH monthly_sales AS (
        SELECT
            DATE_TRUNC('month', order_date):😀ATE AS sales_month,
            SUM(sales) AS total_sales
        FROM superstore.orders
        GROUP BY DATE_TRUNC('month', order_date)
    )

    SELECT
        sales_month,
        total_sales,
        LAG(total_sales) OVER (
            ORDER BY sales_month
        ) AS previous_month_sales,

        total_sales
        - LAG(total_sales) OVER (
            ORDER BY sales_month
```

Monthly Growth %

```
    WITH monthly_sales AS (
        SELECT
            DATE_TRUNC('month', order_date):😀ATE AS sales_month,
            SUM(sales) AS total_sales
        FROM superstore.orders
        GROUP BY DATE_TRUNC('month', order_date)
    ),

    sales_growth AS (
        SELECT
            sales_month,
            total_sales,
            LAG(total_sales) OVER (
```

```
            ORDER BY sales_month
        ) AS previous_month_sales
    FROM monthly_sales
)
```

---

14. Calculate year-over-year sales growth

```
WITH yearly_sales AS (
    SELECT
        year,
        SUM(sales) AS total_sales
    FROM superstore.orders
    GROUP BY year
),

yearly_growth AS (
    SELECT
        year,
        total_sales,
        LAG(total_sales) OVER (
            ORDER BY year
        ) AS previous_year_sales
    FROM yearly_sales
)
```

---

15. Calculate each category's percentage contribution to total sales

```
SELECT
    category,
    SUM(sales) AS category_sales,
    ROUND(
        SUM(sales)
        / SUM(SUM(sales)) OVER () * 100,
        2
    ) AS sales_contribution_percentage
FROM superstore.orders
GROUP BY category
ORDER BY category_sales DESC;
```

---

16. Calculate each region's percentage contribution to total sales

```sql
SELECT
    region,
    SUM(sales) AS region_sales,
    ROUND(
        SUM(sales)
        / SUM(SUM(sales)) OVER () * 100,
        2
    ) AS sales_contribution_percentage
FROM superstore.orders
GROUP BY region
ORDER BY region_sales DESC;
```

---

17. Find cities whose sales are above the average city sales

```sql
WITH city_sales AS (
    SELECT
        city,
        SUM(sales) AS total_sales
    FROM superstore.orders
    GROUP BY city
)

SELECT
    city,
    total_sales
FROM city_sales
WHERE total_sales > (
    SELECT AVG(total_sales)
    FROM city_sales
)
ORDER BY total_sales DESC;
```

---

18. Find products whose sales are above the average product sales

```
WITH product_sales AS (
    SELECT
        product_name,
        SUM(sales) AS total_sales
    FROM superstore.orders
    GROUP BY product_name
)

SELECT
    product_name,
    total_sales
FROM product_sales
WHERE total_sales > (
    SELECT AVG(total_sales)
    FROM product_sales
)
ORDER BY total_sales DESC;
```

---

19. Find the top-performing sub-category within each category

```
WITH subcategory_sales AS (
    SELECT
        category,
        sub_category,
        SUM(sales) AS total_sales
    FROM superstore.orders
    GROUP BY category, sub_category
),

ranked_subcategories AS (
    SELECT
        category,
        sub_category,
        total_sales,
        RANK() OVER (
```

```
                PARTITION BY category
                ORDER BY total_sales DESC
```

---

20. Find the highest-selling product for each region

```
WITH regional_products AS (
    SELECT
        region,
        product_name,
        SUM(sales) AS total_sales,
        ROW_NUMBER() OVER (
            PARTITION BY region
            ORDER BY SUM(sales) DESC
        ) AS rn
    FROM superstore.orders
    GROUP BY region, product_name
)

SELECT
    region,
    product_name,
    total_sales
FROM regional_products
```

---

21. Rank states within each region based on sales

```
SELECT
    region,
    state,
    SUM(sales) AS total_sales,
    RANK() OVER (
        PARTITION BY region
        ORDER BY SUM(sales) DESC
    ) AS state_rank
FROM superstore.orders
GROUP BY region, state
ORDER BY region, state_rank;
```

---

22. Identify the top-performing customer segment in each region

```sql
WITH segment_sales AS (
    SELECT
        region,
        segment,
        SUM(sales) AS total_sales
    FROM superstore.orders
    GROUP BY region, segment
),

ranked_segments AS (
    SELECT
        region,
        segment,
        total_sales,
        RANK() OVER (
            PARTITION BY region
            ORDER BY total_sales DESC
        ) AS segment_rank
```